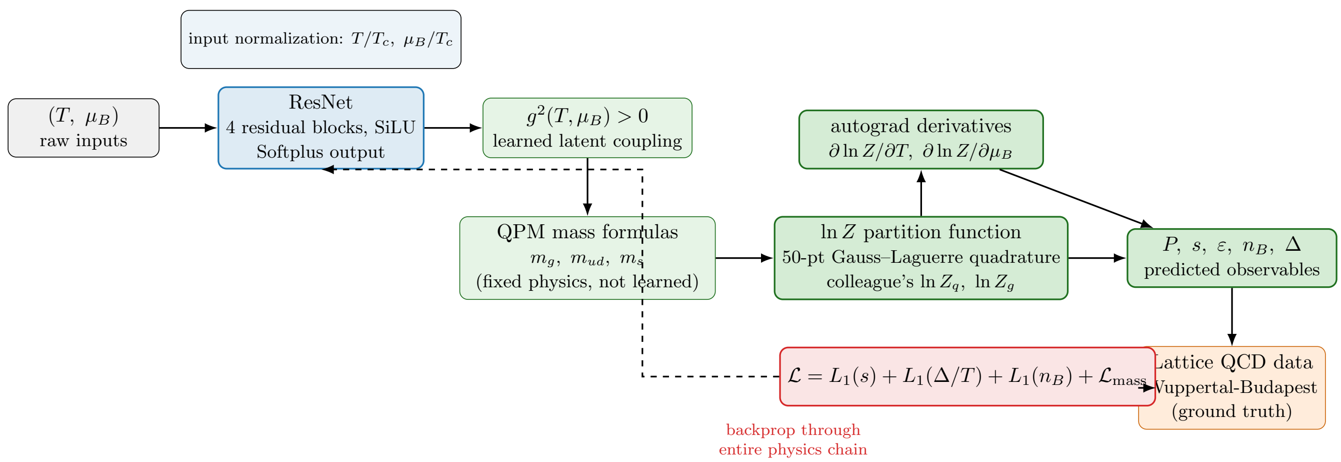
QPM–PINN Architecture

Physics-Informed Neural Network for QGP Thermodynamics

1. Full Pipeline Diagram



Key point: the network learns only $g^2(T, \mu_B)$ (blue box). Every downstream step (green boxes) is a fixed, differentiable, non-learnable physics formula. Gradients flow backward through the *entire physics chain* (dashed arrow) via autograd, so the network is trained end-to-end even though only one scalar is actually learned.

2. Network Architecture — Layers & Parameters

The network (`QPMPinn` in `qgp_pinn.py`) is the *only* learnable component of the whole pipeline. Everything after its output is fixed physics (Secs. 3–5).

Stage	Details
Input	$(T, \mu_B) \in \mathbb{R}^2$, normalized as T/T_c, μ_B/T_c before entering the network (raw, unnormalized T, μ_B are kept aside and passed separately into the physics formulas).
Encoder	`Linear(2 → 64)` → `SiLU`
Residual blocks ×4	Each block: `LayerNorm(64)` → `SiLU` → `Linear(64→64)` → `LayerNorm(64)` → `SiLU` → `Linear(64→64)` → add residual. Second linear layer of each block is zero-initialized so blocks start as near-identity maps.
Output head	`LayerNorm(64)` → `SiLU` → `Linear(64→1)`
Output activation	`Softplus` ($\beta = 1$) — guarantees $g^2 > 0$ while remaining smooth (needed since gradients must flow through g^2 all the way to the loss).
Total trainable parameters	**34,689**

3. Mass Formulas (QPM, HTL-motivated)

$$m_g^2 = \frac{g^2}{6}\left[\left(N_c + \frac{N_f}{2}\right)T^2 + \frac{N_c}{2\pi^2}\sum_i \mu_i^2\right] \tag{1}$$

$$m_{ud}^2 = \frac{N_c^2 - 1}{8N_c}\, g^2 \left[T^2 + \frac{\mu_{ud}^2}{\pi^2}\right] \tag{2}$$

$$m_s^2 = m_{0s}^2 + \frac{N_c^2 - 1}{8N_c}\, g^2 \left[T^2 + \frac{\mu_s^2}{\pi^2}\right] \tag{3}$$

with constants $N_c = 3$, $N_f = 3$, $m_{0s} = 0.0935$ GeV, and chemical potentials

$$\mu_u = \mu_d = \frac{\mu_B}{3}, \qquad \mu_s = \pm\frac{\mu_B}{3} \quad \text{(convention-dependent, see Sec. 9)} \tag{4}$$

4. Partition Function

$$\ln Z_q = (\text{dof}) \cdot \frac{1}{2\pi^2}\int_0^\infty p^2\, dp \Big[\ln\big(1 + e^{-(E-\mu)/T}\big) + \ln\big(1 + e^{-(E+\mu)/T}\big)\Big] \tag{5}$$

$$\ln Z_g = (\text{dof}) \cdot \frac{1}{2\pi^2}\int_0^\infty p^2\, dp\, \ln\big(1 - e^{-E/T}\big) \tag{6}$$

$$E = \sqrt{p^2 + m^2} \tag{7}$$

evaluated via 50-point **Gauss–Laguerre quadrature** (exact for polynomials up to degree 99 against weight e^{-x}; ideal for semi-infinite, exponentially-decaying integrands).

Degrees of freedom:

Species	dof	breakdown
u, d quarks	12	spin(2) × particle/antiparticle(2) × color(3)
s quark	6	spin(2) × color(3)
gluon	16	color(8) × polarization(2)

5. Thermodynamic Relations

$$P = T \ln Z_{\text{tot}} \qquad \text{(pressure)} \tag{8}$$

$$s = T\frac{\partial \ln Z_{\text{tot}}}{\partial T} + \ln Z_{\text{tot}} \qquad \text{(entropy density)} \tag{9}$$

$$n_B = T\frac{\partial \ln Z_{\text{tot}}}{\partial \mu_B} \qquad \text{(net baryon density)} \tag{10}$$

$$\varepsilon = Ts - P + \mu_B n_B \qquad \text{(energy density)} \tag{11}$$

$$\Delta = \varepsilon - 3P \qquad \text{(trace anomaly / interaction measure)} \tag{12}$$

All derivatives computed via **automatic differentiation** (`torch.autograd.grad`) on the analytic $\ln Z_{\text{tot}}(T, \mu_B)$ — exact thermodynamic derivatives, not finite-difference approximations.

6. Loss Function

$$\mathcal{L} = L_1\big(s_{\text{pred}}, s_{\text{lattice}}\big) + L_1\left(\frac{\Delta_{\text{pred}}}{T}, \frac{\Delta_{\text{lattice}}}{T}\right) + L_1\big(n_{B,\text{pred}}, n_{B,\text{lattice}}\big) + \mathcal{L}_{\text{mass}} \tag{13}$$

where the mass-consistency regularizer, active only for $T > 0.8\,T_c$, softly anchors mass ratios to the asymptotic-freedom limit:

$$\mathcal{L}_{\text{mass}} = \beta_1 \left|\frac{m_g}{m_{ud}} - R_{ud}\right| + \beta_2 \left|\frac{m_g}{m_s} - R_s\right|, \qquad \beta_1 = \beta_2 = 0.01 \tag{14}$$

$$g_{\text{ref}}^2 = \frac{48}{27} \cdot \frac{\pi^2}{\ln\big[(\lambda(T/T_c - T_{sr}))^2\big]}, \qquad \lambda = 2.6,\ T_{sr} = 0.57 \tag{15}$$

7. Training Configuration

Setting	**Value**
Optimizer	AdamW, $\beta = (0.9, 0.999)$
Learning rate	1×10^{-3} initial
LR schedule	`StepLR`: ×0.92 every 2000 epochs
Epochs	5000
Train/test split	80% / 20%, `random_state=42`
Checkpoint selection	lowest *training* loss epoch (val loss logged, never back-propagated or used for selection)
Data file	`Thermo_data_central.csv`

8. Dataset

`Thermo_data_central.csv` contains Wuppertal-Budapest lattice QCD equation-of-state data at central (best-fit) values. Key columns: `T` (GeV), `MuB` (GeV), `P/T^4`, `s/T^3`, `E/T^4`, `TraceA`, `nB/T^3` — all dimensionless ratios that get multiplied by the appropriate power of T to recover dimensionful targets during training (see Sec. 11, `s_true` etc.).

9. Open Physics Question: μ_s Sign Convention

	$\mu_s = +\mu_B/3$ (used)	$\mu_s = -\mu_B/3$ (alternative)
Basis	naive baryon-number assignment (s carries +1/3 like u,d)	strangeness-neutrality constraint (standard in heavy-ion collisions)
Source	colleague's reference code	original `qgp_pinn.py`

10. Domain of Validity

Model trained and evaluated only for $T/T_c \gtrsim 1.0$ (deconfined phase). The quasi-particle framework is not physically meaningful below T_c, where matter is confined into hadrons.

11. Results

Evaluated on the held-out 20% test split (`Test_Pipeline_Final.py`):

Observable	**TrueMAPE%**	**GRE%**	$\mathbf{R^2}$	**Bias%**
Pressure (P)	0.40	0.08	1.000	0.29
Energy Density (ε)	0.42	0.09	1.000	−0.24
Entropy (s)	0.35	0.09	1.000	−0.11
Trace Anomaly (Δ)	1.14	0.48	0.999	−1.08
Baryon Density (n_B)	0.86	0.15	1.000	−0.41

Average TrueMAPE across all five targets: **0.63%**.

12. Comparison to Literature (g vs. T/T_c)

The network's inferred $g(T, \mu_B{=}0)$ was compared against digitized Wuppertal-Budapest and hotQCD curves. Same qualitative trend (monotonically decreasing with T, same order of magnitude), but exact values differ. **This is expected, not an error**: g is a latent variable never directly targeted by the loss — it is inferred only indirectly through matching thermodynamics. Different groups' quasi-particle formulations (different DOF counting, mass formula normalization, μ_s convention) can each fit the same lattice thermodynamics essentially perfectly while inferring numerically different $g(T)$.

13. Known Structural Finding

Earlier experiments (using an independent thermodynamics implementation, `qgp_thermodynamics.py`) varied loss weights across several configurations and found that total error never dropped below ∼17% average across P, s, n_B, Δ regardless of reweighting — pushing weight toward n_B improved it to <1% but degraded s and P substantially. This suggests a single shared scalar $g^2(T, \mu_B)$ feeding all three mass formulas may be structurally limited in simultaneously satisfying both T-driven (s, P) and μ_B-driven (n_B, Δ) observables. Under the current pipeline (colleague's exact DOF/$\ln Z$/loss, Sec. 11 results), this constraint appears less severe — worth noting both results if raised.

14. Provenance — What Is Copied vs. Modified

Per direct instruction, this pipeline uses the colleague's thermodynamics engine, DOF counting, loss structure, and μ_s convention **verbatim**. The *only* modification: colleague's original code had three independent networks each directly predicting one mass (`Net1/Net2/Net3`); here, a single ResNet predicts g^2, and QPM mass formulas (Sec. 3) compute the three masses from that one shared value.

QPM–PINN Codebase README

Usage Guide, Function Reference, Variable Dictionary

1. Requirements

- Python 3.10
- `torch` (PyTorch)
- `numpy`
- `pandas`
- `scikit-learn`
- `matplotlib`

Install with:

```
pip install torch numpy pandas scikit-learn matplotlib
```

2. File Structure & Run Order

File	Purpose
`qgp_pinn.py`	Defines the ResNet architecture (`QPMPinn`) that predicts $g^2(T, \mu_B)$.
`train_final.py`	Trains the model on `Thermo_data_central.csv`. Saves checkpoint + loss history to `./model/`.
`plot_results_final.py`	Loads the trained checkpoint, sweeps over T and μ_B/T, produces the 8-panel equation-of-state figure and a predictions CSV.
`Test_Pipeline_Final.py`	Evaluates the trained checkpoint on the held-out 20% test split; reports TrueMAPE, GRE, R^2, Bias%.
`plot_loss_curve.py`	Standalone: regenerates the training-convergence plot from saved `.npy` loss history, without retraining.
`kfold_validate.py`	Runs K-fold cross-validation to check that accuracy is stable across different data splits (not a single-split artifact).

Typical run order:

```
python train_final.py
python plot_results_final.py
python Test_Pipeline_Final.py
```

3. Function Reference

`train_final.py`

- `setup_seed(seed)` — seeds `torch`, `torch.cuda`, and `numpy` RNGs for reproducibility.
- `lnZ_q(T, m, mu, eta, device)` — computes the fermionic (quark) log partition function via 50-point Gauss–Laguerre quadrature. `eta` controls the statistics sign convention in the log term.
- `lnZ_g(T, m, eta, device)` — same, for the bosonic (gluon) sector; no chemical potential term (gluons are colorless / carry no baryon number).

- `compute_thermo(T, mu_B, Mud, Ms, Mgluon, device)` — assembles $\ln Z_{\text{tot}}$ from the three species, takes autograd derivatives w.r.t. T and μ_B, and returns the five thermodynamic observables (pressure, entropy, energy density, baryon density, trace anomaly).
- `qpm_masses_from_g2(g2, T, mu_B)` — applies the QPM mass formulas to convert a predicted g^2 into (m_{ud}, m_s, m_g).
- `plot_loss_curve(train_hist, val_hist, save_path)` — saves a log-scale train/val loss curve PNG.
- `Mass_Tmu_train(learning_rate, epochs, path, device, csv_path)` — the main training loop: builds the network, loads data, trains, saves checkpoint + loss curves.

`plot_results_final.py`

- `load_model(path, device)` — loads a saved checkpoint into a fresh `QPMPinn` instance.
- `run_sweep(pinn, T_min, T_max, n_pts, muB_over_T, device)` — evaluates the trained model across a grid of temperatures at a fixed μ_B/T ratio; returns a `SweepResult`.
- `export_sweeps_to_csv(sweeps, filename)` — writes all swept quantities to CSV.
- `make_figure(sweeps, output, dpi)` — builds the full multi-panel comparison figure (thermodynamics + coupling + masses), including literature overlays.

`Test_Pipeline_Final.py`

- `evaluate(pth_file, csv_file, seed)` — reproduces the exact train/test split used during training, runs the model on the held-out 20%, and prints TrueMAPE / GRE / R^2 / Bias% for all five observables.

`kfold_validate.py`

- `train_one_fold(df_train, df_val, epochs, lr, device, fold_idx)` — trains a fresh model on one fold's training portion, evaluates on that fold's held-out portion.
- `main(csv_path, folds, epochs, lr)` — runs all folds, reports mean ± std TrueMAPE per observable, and a g^2 std/mean ratio as a collapse-detection check.

4. Variable Dictionary

Why so many names for the same quantity? Each script (`train_final.py`, `plot_results_final.py`, `Test_Pipeline_Final.py`) was written at a different point and names its local variables independently, even though several refer to the *exact same physical quantity*. The tables below group these aliases together explicitly so nothing is ambiguous.

Physical inputs

Name	Meaning
`T`	Temperature, in GeV.
`mu_B` / `muB`	Baryon chemical potential, in GeV.
`T_c`	Pseudo-critical (crossover) temperature, fixed at 0.155 GeV.
`ratio` / `muB_over_T`	The dimensionless ratio μ_B/T used to define a sweep band (e.g. 0, 1, 2, 3).

Network output & masses

Aliases (same quantity)	Meaning
`g2`, `g2_g`, `g2_v`	The network's predicted $g^2(T, \mu_B)$ — the *one* learned quantity in the whole model. Suffix `_g`/`_v` just distinguishes which script/pass computed it (plotting sweep vs. validation forward pass); the physical meaning is identical.
`Mud`, `m_ud`	Light quark (u, d) thermal mass, GeV.
`Ms`, `m_s`	Strange quark thermal mass, GeV.
`Mgluon`, `m_g`	Gluon thermal mass, GeV.

Thermodynamic outputs

Aliases (same quantity)	Meaning
`pr`, `P_val`, `P_p`	Pressure P, GeV^4. `pr` in `train_final.py`, `P_val` in the plotting sweep, `P_p` in the test-pipeline evaluation.
`s_pred`, `s_val`, `s_p`	Entropy density s, GeV^3.
`ed`, `eps_val`, `e_p`	Energy density ε, GeV^4.
`nd`, `nB_val`, `nB_p`	Net baryon density n_B, GeV^3.
`Delta`, `delta_val`, `d_p`	Trace anomaly $\Delta = \varepsilon - 3P$, GeV^4.

Data & evaluation

Name	Meaning
`s_true`, `D_true`, `nB_true`	Ground-truth lattice values (training split, `df1`) for entropy, trace anomaly, baryon density.
`s_valid`, `d_valid`, `nB_valid`	Same, but for the held-out validation/test split (`df2`).
`loss_tot`	Total training loss (backpropagated).
`loss_totv`	Total validation loss (computed and logged every epoch, **never** backpropagated).
`TrueMAPE`	$\lvert\text{pred} - \text{true}\rvert / \lvert\text{true}\rvert$, averaged, as a percentage. Honest relative error, no floor/padding.
`GRE`	Global Relative Error: $\lvert\text{pred} - \text{true}\rvert / \max(\lvert\text{true}\rvert)$, averaged. Stable even when `true` passes through zero.
`R`^{2}	Coefficient of determination; 1.0 = perfect fit, < 0 = worse than predicting the mean.
`Bias%`	Mean signed error / mean true value; tells you over- vs under-prediction.

Configuration constants (`plot_results_final.py`)

Name	Meaning
`CHECKPOINT_PATH`	Path to the trained model checkpoint, default `./model/pinn_final_central.pth`.
`T_MIN`, `T_MAX`	Temperature sweep range for plotting, in GeV.
`N_POINTS`	Number of points sampled across the sweep.
`RATIOS_TO_PLOT`	List of μ_B/T bands to plot (default $[0, 1, 2, 3]$).
`OUTPUT_PNG`, `OUTPUT_CSV`	Output filenames for the figure and predictions table.

5. Checkpoint Contents

Every saved `.pth` file is a dictionary with:

- `model_state_dict` — the trained network weights.
- `arch` — a dict of architecture hyperparameters (`hidden_width`, `num_blocks`, `dropout`) needed to reconstruct the same `QPMPinn` before loading weights.
- `optimizer` — optimizer state (for resuming training, not needed for inference).
- `epoch`, `loss` — bookkeeping only.